

Tester (QA) Week3 - CI/CD, Test Cases and Bug Report

What is CI/CD?

You can think of CI/CD as an **automated factory assembly line** for software development, designed to eliminate slow and error-prone manual tasks.

- **CI (Continuous Integration): The "Automated Quality Inspector"** When multiple developers work on an app, merging their code often causes conflicts or unexpected crashes. Continuous Integration solves this by forcing automation. Every time a developer pushes code to a repository (like GitHub), a cloud server instantly wakes up, compiles the code, and **automatically runs your Selenium and Postman testing scripts**. If the tests pass, the new code is safely "integrated" into the main project. If any test fails, the system blocks the merge and alerts the team immediately, catching bugs before they snowball.
- **CD (Continuous Delivery / Deployment): The "Automated Delivery Truck"** Once the CI pipeline verifies that the code is 100% bug-free and all tests have passed, Continuous Deployment takes over. It automatically packages the finished software and pushes it straight to the live production server. This means new features or bug fixes reach the end-users instantly and reliably, with zero manual clicking required by an operations team.

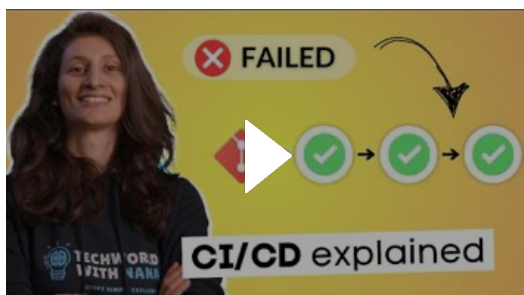
Popular CI/CD Tools

- **GitHub Actions:** Natively built into GitHub, configured via `.github/workflows/*.yml`. It provides managed cloud runners and a vast marketplace of pre-built actions, making it the top choice for modern GitHub-hosted codebases.
- **Jenkins:** The industry-standard open-source automation server. Highly extensible through thousands of plugins, it remains dominant in large enterprises requiring self-hosted, custom on-premise infrastructure.
- **GitLab CI/CD:** A fully integrated DevOps platform embedded in GitLab, driving seamless automated testing, container registry integration, and deployments using `.gitlab-ci.yml`.
- **CircleCI:** A cloud-native SaaS platform optimized for execution speed, flexible caching, and advanced parallel testing workflows.
- **Argo CD:** A declarative GitOps continuous delivery engine built specifically for Kubernetes, maintaining synchronization between Git repositories and live clusters.

Tutorial

CI/CD Explained: The DevOps Skill That Makes You 10x More Valuable

[CI/CD Explained: The DevOps Skill That Makes You 10x More Valuable](#)



Study Note

Study Note

● What is CI/CD with Real-World Examples

CI/CD stands for **Continuous Integration** and **Continuous Delivery (or Continuous Deployment)**, forming the core technical backbone of the DevOps philosophy. It is the practice of converting manual, error-prone software deployment workflows into a streamlined, automated pipeline.

- The Manual Mess Scenario: Without CI/CD, developers manually edit configuration files (such as docker-compose or Kubernetes YAML manifests). Senior engineers SSH directly into servers or run kubectl commands to restart containers and grant access to testers. Deploying to production involves intense stress and anxiety, with teams actively avoiding Friday deployments out of fear that weekend outages will go unmanaged.
- The CI/CD Real-World Solution: Human deployment processes are time-constrained and error-prone. CI/CD replaces these manual interventions with automated pipelines that automatically test, build, and deploy code changes every time code is committed or merged.

● What Problems Does CI/CD Solve

- Eliminates Human Bottlenecks: Replaces manual SSH connections, manual server restarts, and human monitoring with automated scripts.
- Prevents Entangled Bugs & Merge Conflicts: Prevents the chaos of merging large feature branches simultaneously, which previously made untangling broken code almost impossible.
- Reduces Release Anxiety: Replaces high-stakes, stressful release cycles with routine, small, and predictable automated deployments.
- Cuts Debugging Overhead: Catches syntax errors, broken integrations, and security vulnerabilities immediately after code is written rather than right before going live.

● How CI/CD Works

CI/CD works by shifting testing and validation "left" in the development lifecycle. Instead of waiting for a final pre-release testing session, CI/CD executes checks continuously at every step:

1. Commit Level: Automated unit tests and static code reviews run on feature branches every time a developer commits code.
2. Merge Level: Once merged into the main branch, build tools automatically package the application into deployable artifacts (such as version-tagged Docker images) and push them to a container registry.
3. Environment Level: The pipeline automatically deploys the artifact to Development and Staging environments, triggering extensive end-to-end and integration tests.
4. Production Level: The pipeline handles live production deployment using safe, automated rollout strategies.

● What is Continuous Integration (CI)

Continuous Integration (CI) is the practice where developers frequently commit and push small code changes to a shared repository, with automated builds and tests running on every commit.

- Key Mechanics:

- Developers push smaller changes more frequently rather than holding code locally for days.
- Automated unit, functional, and security tests run on individual feature branches before merging.
- AI-powered PR review tools (such as CodeRabbit) can analyze pull requests to catch bugs, security issues, and performance flaws early.
- Primary Benefits:
 - Faster Feedback Loop: Catches issues immediately, making them simple and isolated to fix while the developer is still working on the feature.
 - Smooth Merges: Eliminates massive merge conflicts across team members.

● What is Continuous Delivery

Continuous Delivery automates the release process up to staging and production readiness. It ensures that every code change merged into the main branch is automatically compiled into a release-ready artifact and validated in running environments.

- Automated Environment Testing: Automatically deploys code to Dev and Staging environments to run end-to-end integration tests (validating database connectivity, infrastructure configuration, UI actions, and performance).
- The Human Approval Gate: In Continuous Delivery, deploying to live production is kept behind a single manual approval (e.g., a one-click button trigger). This allows product managers or business stakeholders to demo features or verify final approvals before going live.

● Continuous Deployment vs. Continuous Delivery

Feature	Continuous Delivery	Continuous Deployment
Automation Level	Automated up to Staging/Prod readiness.	Fully automated end-to-end all the way to live Production.
Production Release Trigger	Manual gate: Requires a single human approval/click.	Zero human intervention: Releases automatically if all tests pass.
Typical Use Case	Apps requiring business demo, stakeholder sign-off, or compliance checks.	Highly mature SaaS products with extensive automated test coverage.

● How Does a Real CI/CD Pipeline Look Like

A standard production CI/CD pipeline consists of sequential automated stages:

1. Code Commit & Push: Developer pushes small code changes to a feature branch.
2. CI Validation: Automated unit tests, linting, security scans, and PR checks execute on the branch.
3. Branch Merge & Artifact Build: After passing tests, code merges into the main branch. Automation tools (e.g., GitHub Actions, Jenkins, GitLab CI) build a Docker image, assign a version tag, and push it to a Docker registry.
4. Dev Environment Deployment: The pipeline automatically deploys the new Docker container to the Dev environment.
5. Integration & End-to-End Testing: Automated tests interact with the running application,

testing database connections, external service integrations, and UI workflows.

6. Staging Deployment & Validation: The artifact deploys to Staging, running performance, security, and compliance test suites.
7. Production Deployment: Triggered either via manual approval (Delivery) or automatically (Deployment) using progressive release strategies.
8. Automated Monitoring: Post-deployment monitoring continuously checks system metrics and alerts on anomalies.

● CI/CD and Common Deployment Strategies

To eliminate downtime and production release anxiety, CI/CD pipelines utilize automated deployment strategies:

1. Canary Deployment

- Concept: A progressive rollout that splits live traffic between the current production version and the newly deployed version.
- How It Works:
 - The new application version is initially deployed to a small subset of servers/nodes (e.g., handling 1% to 5% of user traffic).
 - Automated monitoring observes system metrics and error logs for a set duration.
 - If stable, traffic is progressively increased (e.g., to 10%, 20%, and eventually 100%).
- Key Advantage: Reduces the "blast radius"—if a bug reaches production, it only impacts a tiny percentage of users before being caught and rolled back.

2. Blue-Green Deployment

- Concept: Maintains two identical physical or virtual production environments—Blue (active, running current version) and Green (idle, running new version).
- How It Works:
 - The pipeline deploys the new application version to the Green environment while all live user traffic remains directed to Blue.
 - Automated tests validate the Green environment.
 - Once verified, traffic is instantly switched at the router/load-balancer level from Blue to Green.
- Key Advantage: Zero downtime and instant rollback—if an issue occurs in the Green environment post-launch, traffic can be routed immediately back to Blue.

Software Testing: Bug Reports and Test Cases

Software Testing Tutorial #30 - What is a Test Case

[Software Testing Tutorial #30 - What is a Test Case](#)



Study Note:

What is a Test Case?

1. Definition of a Test Case

- A test case is a set of steps or a procedure written to verify whether a specific functionality of a software application works as per the requirements specified by the customer.
- At its core, a test case defines the input actions to be performed on the application and the expected response or outcome that should result.

2. Key Components & Sections of a Test Case

To structure testing effectively, a test case contains several essential sections:

- Title: A concise summary stating what functionality is being tested.
 - *Example:* "Verify successful login with valid email and password" for an e-commerce site.
- Description: A detailed explanation expanding on what the test case covers in the application.
- Prerequisites (Pre-conditions): Any conditions or setup required before executing the test steps.
- Input Actions / Test Steps: The step-by-step actions or inputs a tester must follow on the application.
- Expected Outcome / Response: The correct response or behaviour the application should exhibit after executing the steps.

3. Who Writes Test Cases?

- Test cases are created by members of the software testing/QA team, including test engineers, QA engineers, and test leads.

4. Importance & Benefits of Test Cases

Documenting test cases is critical to the software testing lifecycle for the following reasons:

- Structured Testing Approach: Testing directly without documented test cases is unorganised and risks missing key functionalities or test scenarios.
- Requirement Traceability: Linking test cases to specific requirements ensures that every customer requirement is mapped, tracked, and covered.
- Requirement Coverage & Quality: Mapping test cases highlights any untested requirements so test cases can be added, ensuring high software quality.
- Reusability: Documented test cases serve as reusable test assets that can be executed again in future testing cycles.

[Software Testing Tutorial #31 - Test Scenario Vs Test Case](#)



Test Scenario Vs. Test Case

1. Overview & Core Distinction

In software testing projects, **test scenarios** and **test cases** go hand-in-hand to ensure complete testing coverage¹. The core distinction between them comes down to **what** is being tested versus **how** it is tested:

- **Test Scenario:** A high-level, **one-liner description** specifying **what** functionality or feature is going to be tested²³.
- **Test Case:** A **detailed, step-by-step description** elaborating **how** to test that specific scenario, including specific inputs, execution steps, and expected responses².

2. Derivation & Workflow Hierarchy

Test design follows a structured progression from requirements down to execution steps³⁴:
User Stories / Requirements (SRS) → Test Scenarios → Test Cases

1. **Requirements / User Stories / SRS:** Served as the baseline for creating test scenarios³⁴. If requirements have ambiguities or gaps, test engineers consult with the **Business Analyst (BA)** to clarify them and extend the scenarios⁵.
2. **Test Scenarios:** Derived directly from the requirements to capture all high-level testing ideas³⁴.
3. **Test Cases:** Derived by referring back to the written test scenarios, expanding each into actionable steps³⁴.

3. Comparison Matrix

Aspect	Test Scenario	Test Case
Primary Focus	Defines what to test ² .	Defines how to test ² .
Level of Detail	High-level one-liner summary ²³ .	Low-level detailed steps, prerequisites, and expected outcomes ² .
Source / Derivation	Derived from User Stories / SRS ³⁴ .	Derived from Test Scenarios ³⁴ .
Complexity	Quick to write and outline ⁶ .	Requires more time to detail step-by-step actions ² .

4. Relationship: One-to-One vs. One-to-Many

A test scenario does not always equal a single test case; the mapping depends on the scope of the test⁵⁷:

- **1-to-1 Mapping (Feature Level):** A simple feature scenario may require only one test case⁵⁷.
 - *Example (Registration Feature Scenarios):*⁶
 - Positive Scenario: "Register with email"⁶
 - Positive Scenario: "Register with Google account"⁶
 - Negative Scenario: "Verify registration fails with invalid email"⁶
- **1-to-Many Mapping (End-to-End Scenarios):** A high-level End-to-End (E2E) workflow scenario often requires a **combination of multiple sequential test cases** to cover the complete user journey⁷⁸.
 - *Example (E2E E-Commerce Scenario):* "Customer should be able to purchase items successfully"⁸.
 - *Required Test Cases combined sequentially:*⁴⁸

1. Register new user8
2. Login with registered user8
3. Add items to cart8
4. Proceed to checkout8
5. Make payment8
6. Order confirmation

[Software Testing Tutorial #32 - How to Write Test Cases](#)



How to Write Test Cases

1. Overview & Context

- **Core Objective:** Demonstrates how to design and write effective test cases for software testing projects using a standardized test case template¹.
- **Template Demonstration Method:** The tutorial utilizes an **Excel sheet template** to explain the essential test case fields¹.
- **Excel vs. Test Management Tools:**
 - While smaller organizations or teams with budget/technical constraints might use Excel sheets to document test cases, most companies utilize dedicated **Test Management Tools** (e.g., Zephyr, Xray, TestRail, qTest, TestLodge)¹².
 - The fields present in the Excel template directly map and correlate to dynamic fields within any commercial or open-source test management tool¹more_horiz.

2. Key Fields in a Test Case Template

A standard test case template consists of the following critical sections and metadata fields⁴more_horiz:

- **Test Case ID:** A unique identifier for the test case⁴. When using Excel, it is manually prefixed (e.g., TC01, TC_LOG_01); in test management tools, it is auto-generated dynamically⁴⁷.
- **Created By:** The name of the test engineer who authored the test case⁴⁷.
- **Reviewed By / Review Date:** The name of the reviewer (e.g., Test Lead) and the date the test case was reviewed⁴more_horiz.
- **Priority:** The relative importance/criticality of the test case (e.g., **High, Medium, Low**)⁴⁷.
- **Prerequisites (Pre-conditions):** Any pre-existing setup, environment, or software conditions required before executing the test steps (e.g., "Edge browser must be installed")⁴⁹.
- **Title:** A concise, easy-to-understand summary stating what specific functionality is being verified⁵⁹.
- **Description:** An expanded, detailed summary providing additional context for the test case⁵⁹.

- **Step Number & Test Steps:** The sequential step numbers (1, 2, 3...) and the exact operational actions the tester must perform on the application⁵.
- **Test Data:** Specific input values or data sets required to execute a test step⁵.
- **Expected Result:** The correct, intended behavior or response the software should exhibit upon executing a step⁵.
- **Actual Result:** The observed behavior of the application recorded by the tester during execution⁶.
- **Execution Status / Details:** The execution state of the test step or overall test case (e.g., **Pass, Fail, Blocked, On Hold**)⁶.
- **Comments:** Additional notes, failure details, or observations logged by the tester during execution⁶.

3. Practical Example Walkthrough (E-Commerce Category Filtering)

To demonstrate how to populate the template, the video walks through an e-commerce website filtering scenario⁴more_horiz:

- **Scenario:** Verifying that filtering by the Electronics category displays only electronic products on the page⁷.
- **Sample Test Case Documentation:**
 - **Test Case ID:** TC017
 - **Created By:** Tester Name⁷
 - **Priority:** High⁹
 - **Prerequisite:** Edge browser installed⁹
 - **Title:** *Verify that Electronics category filters only electronic items on the Electronics page*⁹
- **Step Execution & Logging:**
 - **Step 1:** Launch the browser and open the website URL $\rightarrow$ **Expected Result:** Website launches successfully¹⁰.
 - **Step 2:** Click on the "Electronics" category link and scroll down $\rightarrow$ **Expected Result:** Only electronic items are displayed¹⁰.
 - **Actual Execution Handling:** If non-electronic items (e.g., items from other categories) appear in the list during live testing, the **Actual Result** is logged as a discrepancy, the **Execution Status** is set to **Fail**, and detailed bug observations are noted under **Comments**⁸¹⁰.

4. Transitioning from Excel to Test Management Tools

- **Drawbacks of Excel Management:** Maintaining test cases in Excel leads to version control issues, tedious review processes, and manual tracking overhead⁸¹¹.
- **QA Leadership Role:** As a QA engineer or lead, it is best practice to recommend transitioning to open-source or commercial test management tools (e.g., Zephyr or Xray plugins for Jira) to improve collaboration, automated reporting, and efficiency²¹¹

[How to Write Bug Report in Software Testing](#)



How to Write a Bug Report in Software Testing

1. Overview & Context

- Bug Tracking System: Jira is the industry-standard project management and bug tracking platform used by QA teams worldwide.
- Jira Workspace Navigation:
 - Summary Dashboard: Provides high-level project activity metrics for Project Managers and QA Leads.
 - Backlog: Contains all user stories, tasks, and future tickets awaiting sprint planning.
 - Board: The primary daily workspace for QA engineers and developers, displaying columns such as *To Do*, *In Progress*, *In QA*, and *Done*.

2. Real-World Bug Demonstration Example

To demonstrate filing a professional bug report, the tutorial identifies a functional defect on an e-commerce/education platform (codemify.com):

- Intended Logic: Clicking any "Try Intro Week" button across the site should redirect users to the main intro course choice page (/intro).
- Discovered Bug: Clicking the "Try Intro Week" button at the bottom of the Reviews page incorrectly redirects the user to the manual QA checkout page (/1week-web-20).

3. Key Fields of a Professional Bug Report

Title / Summary

- The 3-Question Formula: A good summary title must clearly answer three core questions:
 1. What happened? (Incorrect redirection)
 2. Where did it happen? (On the Reviews page)
 3. Under what circumstances? (Upon clicking the "Try Intro Week" button)
- *Example Title:* "Incorrect redirection upon clicking 'Try Intro Week' on Reviews page".

Prerequisites (Pre-conditions)

- Any required user state, account setup, or environment conditions needed before executing test steps.
- *In this example:* N/A (No account login or special setup required).

Steps to Reproduce (STR)

- Clear, numbered, step-by-step actions that developers can follow to recreate the bug.
- Includes direct URLs to save developer time:
 1. Navigate to the Reviews page (<https://codemify.com/reviews>).
 2. Scroll all the way down to the bottom of the page.
 3. Click the yellow "Try Intro Week" button.

Actual Result

- The incorrect behavior observed in the application, including specific URLs or error messages.
- *Example:* User is redirected to the manual intro week sales landing page (/1week-web-20).

Expected Result

- The correct behavior specified by requirements.
- *Example:* User should be redirected to the main intro selection page (/intro).

Attachments / Screenshots

- Visual proof of the defect to assist developers in diagnosing the issue.
- Screenshots should highlight the clicked element and clearly capture the browser address bar / URL.

Metadata & Workflow Fields

- Assignee: The software developer responsible for fixing the defect.
- Reporter: The QA engineer who identified and logged the bug.
- Priority / Severity: Indicates the urgency and impact level of the issue.
- Affected Version: Specifies the build or software release version where the bug exists (e.g., v4.10.0).

4. Sprint Management: Active Sprint vs. Backlog

- Active Sprint Assignment: Assigning the bug ticket to the current active sprint ensures it appears immediately on the team's active Board for developers to fix ASAP.
- Backlog Placement: If no active sprint is selected, the bug report automatically falls into the project Backlog, delaying review until future sprint planning meetings.